A Survey of Efficient LLM Serving with KV Cache, Speculative Decoding, and Quantization

InteractiveSurvey

Abstract

The rapid proliferation of Large Language Models has revolutionized natural language processing, yet their deployment faces severe bottlenecks due to the exponential memory growth of Key-Value caches during inference. This survey provides a comprehensive examination of efficient serving techniques, specifically synthesizing advancements in KV cache compression, speculative decoding, and quantization to address these scalability challenges. We systematically analyze precision reduction strategies, including mixed-precision allocation and outlier mitigation via smoothing, which enable aggressive compression while preserving generative fidelity. Furthermore, the review explores structural sparsity mechanisms and dynamic token eviction policies that exploit attention heterogeneity to reduce memory footprints without compromising long-range dependencies. By integrating low-rank decomposition with integer-only arithmetic frameworks, this work establishes a unified taxonomy that bridges theoretical algorithms and practical system implementations across diverse hardware architectures. Ultimately, this synthesis offers a robust foundation for researchers and practitioners aiming to deploy large-scale models with optimal efficiency and scalability in production environments.

1 Introduction

The rapid proliferation of Large Language Models (LLMs) has revolutionized natural language processing, enabling unprecedented capabilities in reasoning, generation, and complex task automation [1]. However, the deployment of these massive models faces severe bottlenecks, primarily driven by the exponential growth of memory requirements during inference. As context lengths extend to accommodate complex documents and multi-turn conversations, the Key-Value (KV) cache, which stores intermediate attention states, consumes a dominant portion of available High Bandwidth Memory [2]. This memory pressure severely limits batch sizes and throughput, creating a critical disparity between the computational potential of modern accelerators and the actual serving efficiency achievable in production environments. Consequently, optimizing the storage and computation of the KV cache has become a paramount challenge for scalable LLM serving [3].

This survey paper provides a comprehensive examination of efficient LLM serving techniques, with a specific focus on three pivotal optimization paradigms: KV cache compression, speculative decoding, and quantization [4]. While previous works have often addressed these areas in isolation, this review synthesizes recent advancements to illustrate how these mechanisms interact to alleviate memory bandwidth constraints and reduce latency. By concentrating on the structural and numerical properties of attention mechanisms, we analyze how modern systems achieve aggressive compression without compromising generative fidelity. The scope encompasses both algorithmic innovations in precision reduction and system-level strategies for dynamic resource management, offering a unified perspective on the current state of efficient inference.

The initial portion of this review delves into the intricacies of quantization and compression strategies tailored specifically for KV caches [5]. We examine precision reduction techniques, including mixed-precision allocation and bitwise optimization, which balance memory efficiency against accuracy by treating statistical outliers differently from inliers. Furthermore, we explore outlier mitigation via smoothing techniques that mathematically transform activation distributions to enable low-bit integer arithmetic. The discussion extends to integer-only arithmetic frameworks that

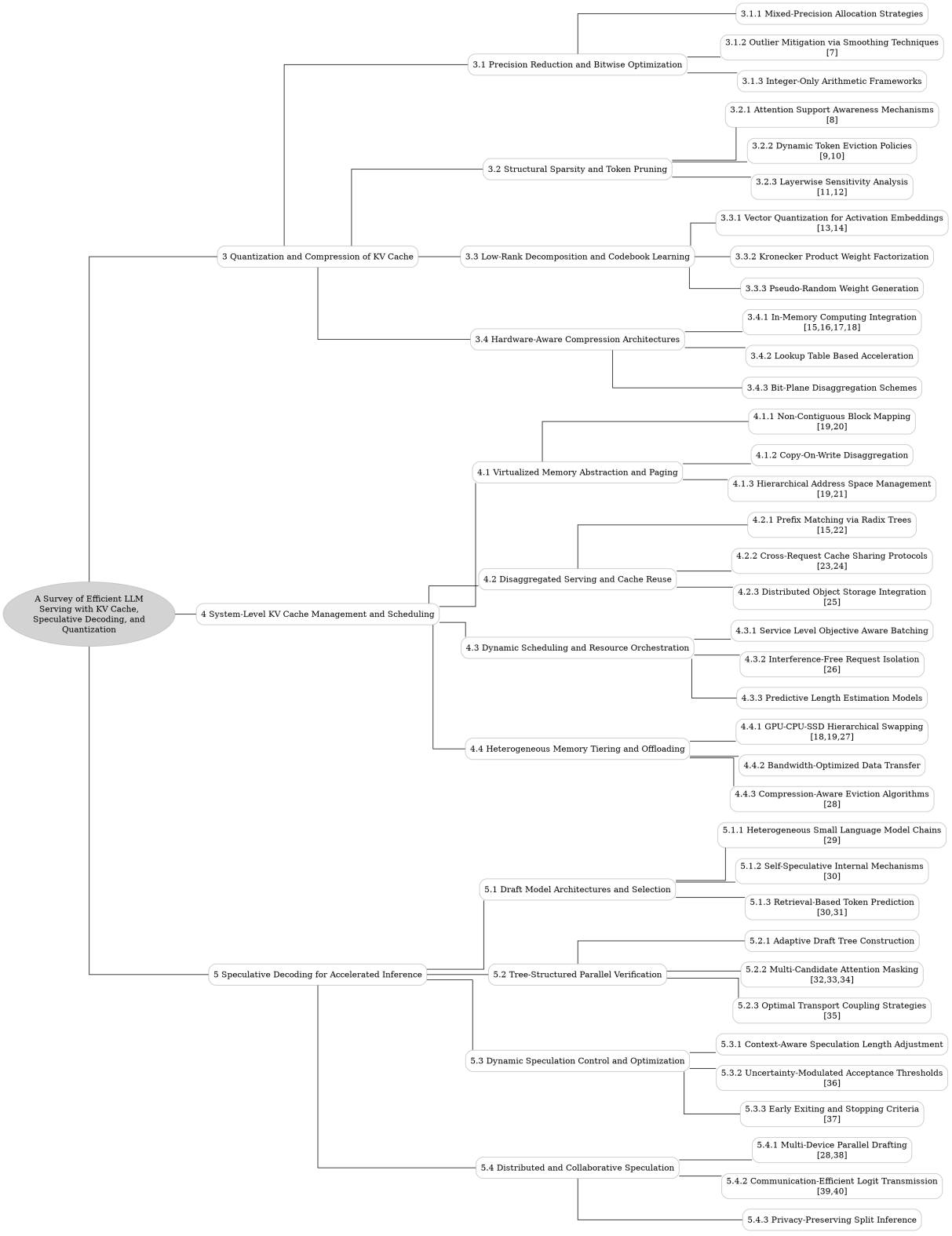

Figure 1: The outline of our survey: A Survey of Efficient LLM Serving with KV Cache, Speculative Decoding, and Quantization

Figure 2: Chart from 'HADES: Hardware Accelerated Decoding for Efficient Speculation in Large Language Models' (arXiv:2412.19925)

Figure 3: Chart from 'SAW-INT4: System-AWare 4-Bit KV-Cache Quantization for Real-World LLM Serving' (arXiv:2604.19157)

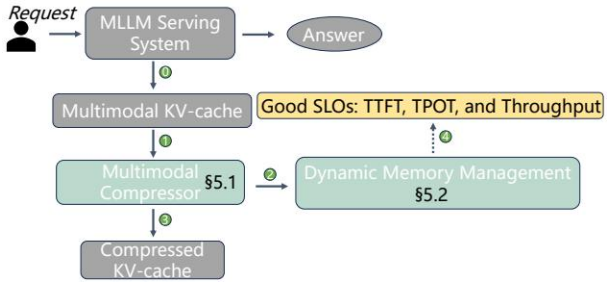

Figure 4: Chart from 'FastCache: Optimizing Multimodal LLM Serving through Lightweight KV-Cache Compression Framework' (arXiv:2503.08461)

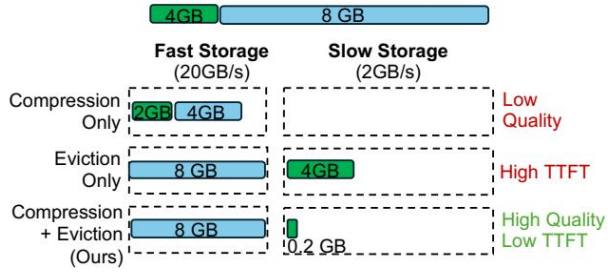

Figure 5: Chart from 'EVICPRESS: Joint KV-Cache Compression and Eviction for Efficient LLM Serving' (arXiv:2512.14946)

eliminate floating-point operations entirely, detailing how affine transformations and fused kernels maintain model performance while maximizing hardware utilization on resource-constrained devices.

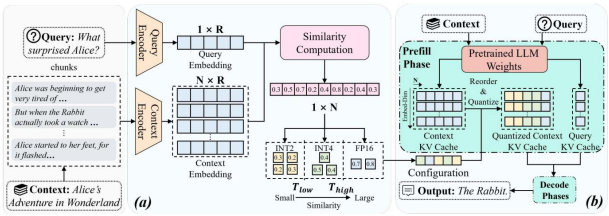

Figure 6: Chart from 'Cocktail: Chunk-Adaptive Mixed-Precision Quantization for Long-Context LLM Inference' (arXiv:2503.23294)

Subsequently, the analysis shifts toward exploiting structural sparsity and implementing dynamic token pruning mechanisms. We investigate attention support awareness systems that distinguish between critical retrieval heads and streaming heads to prioritize salient information dynamically. This section details dynamic token eviction policies that selectively discard less critical KV entries based on real-time attention metrics, alongside layerwise sensitivity analysis which reveals the heterogeneous vulnerability of different network depths to quantization errors. These approaches collectively demonstrate how adaptive resource allocation can significantly reduce memory footprints while preserving the long-range dependencies essential for coherent generation.

Finally, the survey covers advanced compression methodologies rooted in low-rank decomposition and codebook learning. We discuss vector quantization for activation embeddings, which maps continuous high-dimensional vectors to discrete codebook entries to preserve structural correlations often lost in scalar quantization. By integrating these decomposition techniques with sparse residual modeling, modern frameworks achieve ultra-low bit-width representations that maintain high reconstruction fidelity. This holistic overview connects numerical compression, structural sparsity, and algorithmic acceleration, providing a robust foundation for understanding the trade-offs inherent in next-generation serving systems.

The primary contribution of this work lies in its systematic taxonomy of efficient serving techniques, bridging the gap between theoretical compression algorithms and practical system imple-

mentations. Unlike existing surveys that focus broadly on model pruning or distillation, this paper offers a deep dive into the specific mechanics of KV cache optimization, speculative decoding integration, and quantization-aware serving. We critically evaluate the efficacy of various strategies across different hardware architectures and context lengths, identifying key trends and unresolved challenges. By synthesizing these diverse threads, this survey serves as an essential reference for researchers and practitioners aiming to deploy large-scale language models with optimal efficiency and scalability [6].

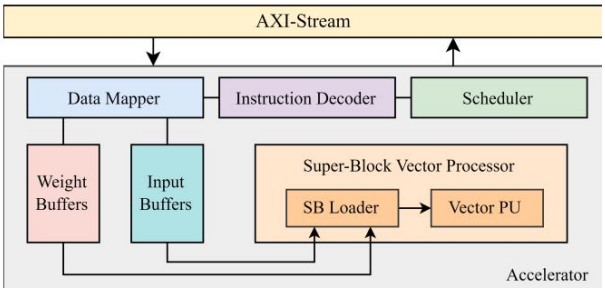

Figure 7: Chart from 'Designing Efficient LLM Accelerators for Edge Devices' (arXiv:2408.00462)

2 Quantization and Compression of KV Cache

2.1 Precision Reduction and Bitwise Optimization

2.1.1 Mixed-Precision Allocation Strategies

Mixed-precision allocation strategies have emerged as a critical mechanism to balance memory efficiency and inference accuracy in large language model serving. These approaches typically assign higher precision datatypes, such as FP16, to statistical outliers while quantizing inliers to lower bit-widths like INT4. Although this differentiation preserves model fidelity, it introduces structural sparsity that incurs significant storage overhead due to the necessity of indexing metadata. Specifically, storing mixed-precision outliers often requires approximately 23 bits per entry to accommodate the high-precision value, index pointers, and group indicators, thereby demanding specialized hardware support to manage the resulting irregular memory access patterns efficiently.

To address the limitations of uniform quantization, recent advancements advocate for fine-grained, asymmetric precision schemes tailored to the distinct distributional characteristics of Key and Value caches. Methods such as KIVI and KVQuant leverage the insight that key channels and value tokens exhibit divergent magnitude patterns, proposing per-channel quantization for keys and per-token or group-wise quantization for values. This adaptive granularity allows for aggressive compression, such as tuning-free 2-bit representations, without the severe accuracy degradation associated with coarse global quantization. By isolating outliers within specific vectors or channels, these strategies minimize reconstruction error while avoiding the computational penalties of global channel reordering.

Beyond static configurations, contemporary research explores dynamic, layer-wise, and chunk-adaptive allocation frameworks to optimize the cost-accuracy Pareto frontier. Techniques like CocktailQuant and hierarchical paging systems adjust bit-widths across different network layers or context segments based on offline calibration or online sensitivity analysis. These methods often employ multi-objective optimization to determine optimal precision pairs that satisfy strict memory budgets while bounding accuracy loss. Furthermore, hybrid approaches integrate mixed-precision KV caches with speculative decoding and memory paging, enabling heterogeneous precision serving that maximizes throughput for long-context workloads without compromising the reasoning capabilities of dense floating-point baselines.

2.1.2 Outlier Mitigation via Smoothing Techniques

Outlier mitigation via smoothing techniques primarily addresses the asymmetrical and concentrated distribution of activation outliers that hinder low-bit quantization in Large Language Models [7]. Methods like SmoothQuant mathematically transfer activation variance to weights through equivalent transformations, enabling efficient INT8 arithmetic by balancing magnitude disparities across channels. Similarly, OS+ introduces channel-wise shifting and scaling parameters, determined via search processes, to specifically alleviate the challenges posed by non-uniform outlier distributions without altering the underlying model output.

Advanced approaches further refine this paradigm by optimizing transformation boundaries and integrating floating-point formats to

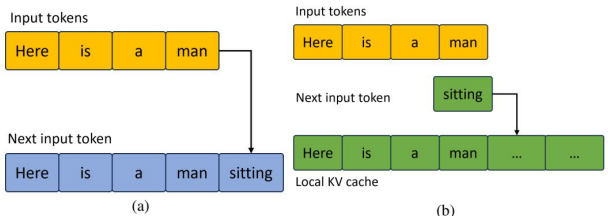

Figure 8: Chart from 'Efficient LLM Inference on CPUs' (arXiv:2311.00502)

enhance representation ranges. Omniquant diverges from empirical designs by jointly optimizing weight clipping boundaries and scaling factors to minimize quantization errors directly. Concurrently, ZeroQuant-FP investigates the feasibility of quantizing activations into FP4 and FP8 formats, demonstrating that floating-point types offer superior resilience against outliers compared to integer counterparts, thereby preserving model fidelity during aggressive compression.

Complementary strategies employ structural modifications and hybrid storage mechanisms to isolate and rectify outlier impacts. QLLM utilizes channel reassembly alongside learnable low-rank approximations to decouple coherent error bases from incoherent outlier noise. Furthermore, techniques like Oaken convert outliers into inliers by shifting values toward smaller ranges prior to quantization, subsequently storing data using dense tensors for inliers while fusing sparse outliers. These methods collectively reduce the magnitude gap between extreme and typical values, ensuring robust performance even under stringent bit-width constraints.

2.1.3 Integer-Only Arithmetic Frameworks

Integer-only arithmetic frameworks have emerged as a critical solution for deploying large language models on resource-constrained hardware by eliminating floating-point operations entirely. These approaches quantize both weights and activations to low-bit integers, typically INT8 or lower, enabling the exclusive use of efficient integer multiply-accumulate units prevalent in modern accelerators. By mapping floating-point values to discrete integer representations via scaling factors and zero-points, these frameworks significantly reduce memory bandwidth requirements and computational latency while maintaining acceptable model accuracy through careful calibration.

To address the inherent precision loss in non-linear operations such as softmax and layer normalization, advanced techniques incorporate equivalent affine transformations and dynamic range adjustment. Methods like AffineQuant extend the optimization scope by introducing transformation invariance, allowing error compensation without reverting to higher-precision arithmetic. Furthermore, recent developments integrate fused dequantization-computation kernels that perform on-the-fly conversion within tiled execution pipelines, thereby minimizing memory access overhead and avoiding the storage of intermediate high-precision matrices.

Despite their efficiency, pure integer-only schemes face challenges in extreme low-bit regimes where non-linear function approximation becomes unstable. Consequently, contemporary research focuses on hybrid strategies that combine integer arithmetic with specialized lookup tables or adaptive granularity quantization across tokens and channels. These enhancements ensure robustness across diverse model architectures and tasks, from mathematical reasoning to long-context generation, while preserving the hardware benefits of integer-only execution. As a result, integer-only frameworks now serve as a foundational component in scalable LLM inference systems, offering a practical balance between performance, memory footprint, and deployment flexibility.

2.2 Structural Sparsity and Token Pruning

2.2.1 Attention Support Awareness Mechanisms

Attention support awareness mechanisms fundamentally address the computational and memory bottlenecks inherent in standard self-attention by exploiting the intrinsic sparsity and heterogeneity of attention distributions [8]. Rather than treating all tokens uniformly, these systems dynamically identify and prioritize salient information, distinguishing between critical retrieval heads responsible for long-range dependencies and streaming heads that process local context. This differentiation enables adaptive resource allocation, where high-importance tokens are retained in the Key-Value cache while less significant entries are evicted or compressed, significantly reducing memory footprint without compromising generative fidelity.

These mechanisms operate through sophisticated gating strategies and sensitivity analysis to

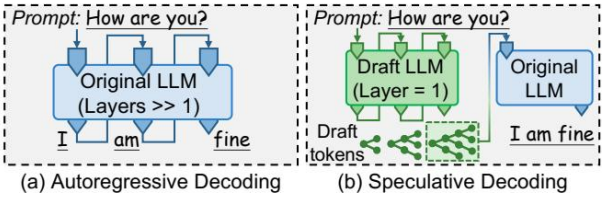

Figure 9: Chart from 'SpecEE: Accelerating Large Language Model Inference with Speculative Early Exiting' (arXiv:2504.08850)

monitor attention score drift across layers and time steps. By quantifying the utility of individual tokens based on their contribution to the final output distribution, the system can implement fine-grained cache management policies that respond to both spatial layer-wise variations and temporal evolution during decoding. Advanced approaches utilize optimization-based identification to assign gating values to each head, effectively creating a hybrid architecture that blends dense attention for crucial features with sparse approximations for redundant data, thereby mitigating the quadratic complexity associated with long-context inference.

Furthermore, awareness mechanisms integrate tightly with hardware-efficient execution kernels to maximize throughput and minimize latency. By aligning software-level sparsity patterns with underlying memory hierarchies, these methods reduce global memory I/O overhead and enable efficient on-chip data reuse during activation-weight operations. The synergy between dynamic token dropping, block-wise computation, and IO-aware scheduling ensures that the asymmetry between compute-bound prefill and memory-bound decode stages is effectively balanced. Consequently, attention support awareness not only accelerates inference but also enhances scalability, allowing large language models to handle extended contexts within constrained hardware resources.

2.2.2 Dynamic Token Eviction Policies

Dynamic token eviction policies address the memory bottleneck in long-context inference by selectively discarding less critical Key-Value (KV) entries based on real-time attention metrics [9]. Unlike static pruning, these mechanisms continuously evaluate token utility as the autoregressive generation unfolds, leveraging the observed sparsity in attention scores to identify candidates for removal [10]. Prominent approaches, such as those utilizing accumulated attention scores, evict tokens with historically low interaction frequencies while preserving "heavy-hitter" tokens that consistently attract high attention weights, thereby maintaining generative fidelity within constrained High Bandwidth Memory limits.

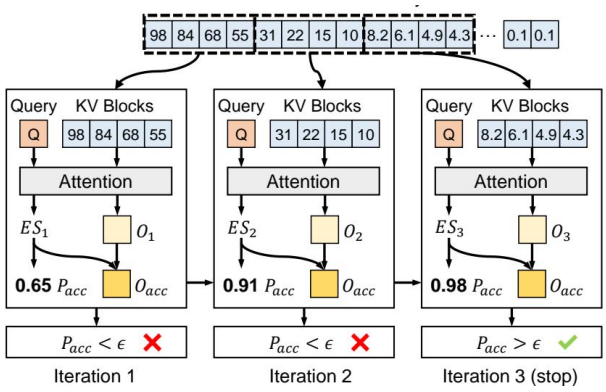

Figure 10: Chart from 'Progressive Sparse Attention: Algorithm and System Co-design for Efficient Attention in LLM Serving' (arXiv:2503.00392)

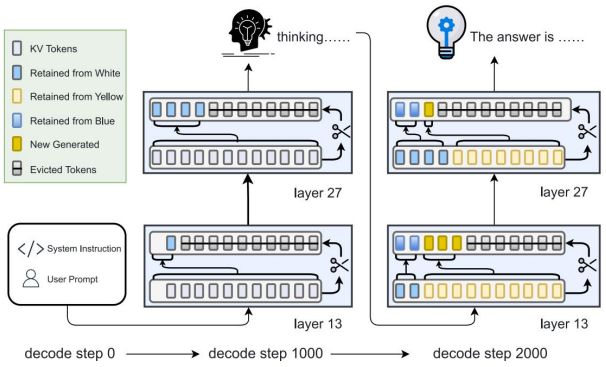

Figure 11: Chart from 'Lethe: Layer- and Time-Adaptive KV Cache Pruning for Reasoning-Intensive LLM Serving' (arXiv:2511.06029)

To mitigate the risk of prematurely discarding tokens that may become relevant in future decoding steps, advanced policies incorporate temporal dynamics and structural constraints into the eviction logic. Systems often enforce the retention of recent tokens and specific boundary anchors to preserve local coherence and global context structure, preventing the degradation of reasoning capabilities in extended sequences. Furthermore, layer-wise sparsity estimators dynamically allocate token budgets across network depths, recognizing that attention distribution varies significantly between early and late layers, allowing for more granular and efficient memory reclamation strategies that adapt to the evolving state of the generation process.

While token-level selection offers theoretical precision in identifying salient information, it frequently incurs prohibitive runtime overhead due to fine-grained metadata management and sorting operations. Consequently, recent innovations have shifted towards block-level eviction schemes that operate on fixed-size memory pages, striking a balance between selection accuracy and computational efficiency. These hybrid designs utilize metadata vectors to represent block contents, enabling rapid identification of evictable regions without scanning individual tokens, thus ensuring that memory savings do not compromise the Time-Between-Tokens latency required for responsive interactive serving.

2.2.3 Layerwise Sensitivity Analysis

Layerwise sensitivity analysis reveals that Large Language Models exhibit heterogeneous vulnerability to quantization errors across different depths. Empirical observations indicate that shallower layers are significantly more sensitive to information loss in Key-Value (KV) cache tensors compared to deeper layers, necessitating a non-uniform quantization strategy [11]. This disparity arises because early layers capture fundamental syntactic and semantic structures where precision degradation disproportionately impacts overall output quality, whereas deeper layers demonstrate greater robustness to coarser quantization bin sizes.

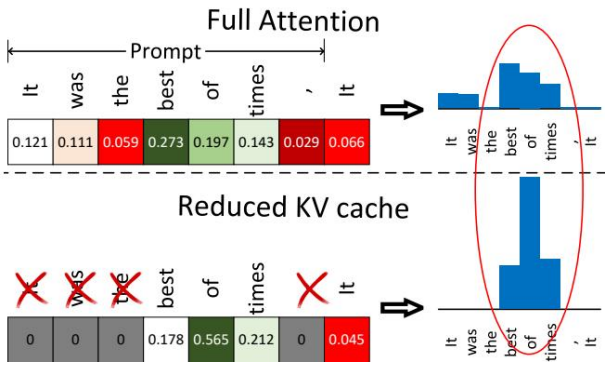

Figure 12: Chart from 'KEYFORMER: KV CACHE REDUCTION THROUGH KEY TOKENS SELECTION FOR EFFICIENT GENERATIVE INFERENCE' (arXiv:2403.09054)

To address this heterogeneity, advanced methodologies employ layer-specific quantization factors rather than global parameters. Techniques such as SmoothQuant utilize reparameterization to migrate activation outliers into weight matrices, allowing for distinct scaling factors per layer that shrink activation ranges while expanding weight channels. Furthermore, statistical profiling of KV cache magnitudes demonstrates that value distributions vary distinctly across decoder layers, driven by unique weight configurations [12]. Consequently, optimal compression schemes determine quantization granularities individually for each model and layer, often segmenting vectors into multiple groups based on value magnitude to minimize reconstruction error.

$$s_p = \sum_i \max(q_i \cdot k_{p,i}^{\min},\ q_i \cdot k_{p,i}^{\max})$$

Figure 13: Chart from 'FLEXICACHE: LEVERAGING TEMPORAL STABILITY OF ATTENTION HEADS FOR EFFICIENT KV CACHE MANAGEMENT' (arXiv:2511.00868)

Recent frameworks extend this analysis to dynamic, context-adaptive precision allocation, particularly within Mixture-of-Experts architectures. By evaluating expert importance dynamically, these systems assign higher bit-widths to critical components in sensitive layers while reducing precision in robust regions. This approach leverages the insight that quantization noise tolerance is not static but depends on the specific layer index and the nature of the input data. Ultimately, layerwise sensitivity analysis provides the theoretical foundation for mixed-precision inference, enabling significant memory savings and latency reductions while maintaining model fidelity through targeted error mitigation.

2.3 Low-Rank Decomposition and Codebook Learning

2.3.1 Vector Quantization for Activation Embeddings

Vector quantization (VQ) for activation embeddings addresses the challenge of compressing high-dimensional, dynamic activation tensors by mapping continuous vectors to discrete codebook entries. Unlike scalar quantization, which operates element-wise, VQ clusters groups of activation values into representative centroids, significantly reducing memory footprint while preserving structural correlations within the data. This approach is particularly effective for activation outliers, as it allows the model to allocate specific codewords to rare but critical activation patterns that uniform

quantization schemes often distort, thereby maintaining inference accuracy in large language models [13].

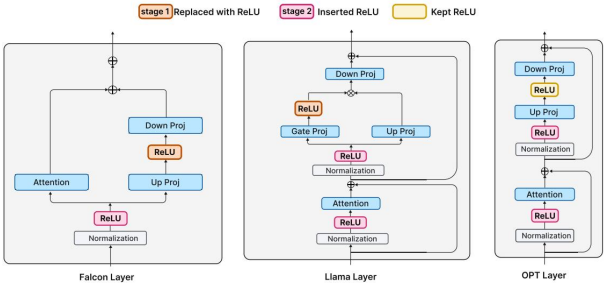

Figure 14: Chart from 'ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models' (arXiv:2310.04564)

Recent advancements have extended VQ beyond static weight compression to include dynamic activation and Key-Value (KV) cache management. Techniques such as Product Quantization partition high-dimensional activation vectors into disjoint sub-vectors, each quantized independently against smaller sub-codebooks, which drastically reduces the computational overhead of codebook search and storage. Furthermore, hybrid strategies combine VQ with low-rank compensation or sparse residual modeling to approximate quantization errors, ensuring that the reconstruction fidelity remains high even at ultra-low bit-widths. These methods often employ per-token or channel-wise granularity to adapt to the heterogeneous distribution of activations across different transformer layers.

Implementing VQ for activations requires careful optimization of the codebook generation and lookup processes to minimize latency during inference. Modern frameworks utilize pre-computed lookup tables for dot-product operations between quantized activations and weights, fusing these steps with accumulation kernels to accelerate the forward pass. While training-aware approaches leverage straight-through estimators to refine codebooks jointly with model parameters, post-training methods focus on efficient clustering algorithms to derive optimal centroids from calibration data. Despite the increased complexity in kernel design compared to scalar quantization, VQ offers a superior trade-off between compression ratio and perplexity, enabling scalable deployment of models with extended context windows [14].

$$Q_i = XW_{Q_i},\ K_{i/g} = XW_{K_{i/g}},\ V_{i/g} = XW_{V_{i/g}},$$
$$O_i = \mathrm{softmax}(Q_i K_{i/g}^T / \sqrt{d}) V_{i/g},$$

Figure 15: Chart from 'LUT-LLM: Efficient Large Language Model Inference with Memory-based Computations on FPGAs' (arXiv:2511.06174)

2.3.2 Kronecker Product Weight Factorization

Kronecker product weight factorization represents a sophisticated tensor decomposition strategy designed to mitigate the memory footprint of large language models by exploiting inherent structural redundancies within weight matrices. Unlike standard low-rank approximations that decompose a matrix into the product of two smaller dense matrices, this approach expresses the original high-dimensional weight tensor as the Kronecker product of two or more significantly smaller factor matrices. This mathematical formulation allows for an exponential reduction in parameter count relative to the dimensions of the factor matrices, effectively compressing the model while preserving the representational capacity required for complex linguistic tasks.

The implementation of this factorization typically involves replacing standard linear projection layers with specialized operations that compute the output via the properties of the Kronecker product, thereby avoiding the explicit materialization of the full uncompressed weight matrix during inference. By decomposing feed-forward network weights and attention projections into global shared tensors capturing common information and local auxiliary tensors for specialized features, the method achieves a balance between compression ratio and accuracy. This structure is particularly advantageous for hardware acceleration, as the resulting smaller factor matrices can be optimized for cache locality and parallel processing, reducing memory bandwidth bottlenecks that often plague large-scale transformer deployments.

Despite its theoretical efficiency, applying Kronecker product factorization requires careful initialization and fine-tuning to prevent performance degradation, as the constrained parameter space may limit the model's ability to learn arbitrary transformations. Recent advancements integrate this factorization with quantization techniques and activation-aware scaling to further enhance com-

pression rates without sacrificing perplexity scores. The synergy between Kronecker decomposition and sparse structures enables a flexible trade-off between computational complexity and storage requirements, making it a viable candidate for deploying massive models on resource-constrained edge devices while maintaining the fidelity of full-precision counterparts.

2.3.3 Pseudo-Random Weight Generation

Pseudo-random weight generation serves as a critical mechanism in advanced quantization frameworks to approximate the statistical properties of full-precision matrices without storing explicit values. Unlike deterministic pruning or fixed-codebook approaches, this technique leverages seeded stochastic processes to reconstruct weight distributions on the fly, significantly reducing memory footprint. By generating weights dynamically during the forward pass, systems can bypass the latency associated with loading large static tensors from off-chip memory, effectively trading computational cycles for bandwidth savings.

The implementation typically involves mapping low-bit indices to high-dimensional vectors through a deterministic pseudo-random number generator (PRNG) synchronized across processing units. This approach ensures that the generated weights maintain the expected mean and variance of the original distribution, which is essential for preserving model accuracy post-quantization. Furthermore, when integrated with activation-aware schemes, the generation process can be conditioned on input statistics to adaptively scale the pseudo-random outputs, thereby mitigating the impact of outliers that often degrade performance in uniform quantization settings.

Recent advancements have extended this paradigm to support fine-grained group-wise generation, allowing for heterogeneous precision within a single layer. By combining pseudo-random synthesis with residual correction terms, modern algorithms can iteratively refine the approximation error, achieving near-lossless compression ratios. This method proves particularly effective in scenarios where storage constraints are paramount, as it eliminates the need for large lookup tables while maintaining the structural integrity required for stable gradient propagation during potential fine-tuning phases.

2.4 Hardware-Aware Compression Architectures

2.4.1 In-Memory Computing Integration

In-memory computing integration addresses the memory bandwidth bottleneck in Large Language Model (LLM) inference by shifting computational logic closer to or directly into storage arrays [15]. Unlike traditional von Neumann architectures that suffer from excessive data movement between processing units and high-bandwidth memory, this paradigm leverages Processing-in-Memory (PIM) to execute matrix operations within the memory fabric itself. Recent architectural proposals unify Neural Processing Units with PIM-enabled memory systems, allowing for end-to-end inference operations where memory controllers simultaneously handle data retrieval and arithmetic execution. This approach significantly reduces latency and energy consumption by eliminating the need to transfer massive weight matrices and Key-Value (KV) caches across the interconnect.

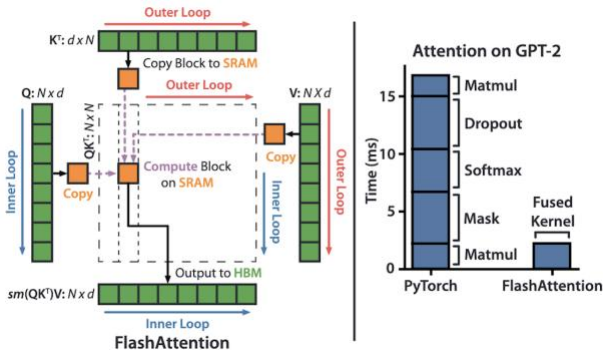

Figure 16: Chart from 'Inference Optimization of Foundation Models on AI Accelerators' (arXiv:2407.09111)

To maximize the efficacy of in-memory architectures, advanced memory management techniques are employed to optimize data representation and access patterns. Research demonstrates the utility of storing model parameters in compressed formats, such as Compressed Sparse Row (CSR), alongside dynamic bitwidth assignment solved via integer programming to balance precision and resource constraints. Furthermore, specialized hardware-aligned kernels are designed to fuse dequantization operators directly with attention computations. By integrating these operations, systems can process low-bit quantized KV caches without incurring the substantial overhead of explicit dequantization steps, thereby minimizing global memory accesses and enhancing

throughput during autoregressive decoding [16].

$$p_\theta^{\mathrm{Rec}} = p_\theta^{\mathrm{Hier}}.$$

Figure 17: Chart from 'PHOTON: Hierarchical Autoregressive Modeling for Lightspeed and Memory-Efficient Language Generation' (arXiv:2512.20687)

The implementation of these systems often relies on asynchronous pipeline execution to hide remaining data transfer latencies. By leveraging APIs that overlap memory transfers with computation, quantized data can be streamed from global memory to shared buffers while parallel compute cores process preceding tiles. This strategy is complemented by adaptive memory management schemes that dynamically swap unused KV cache segments and apply compression-aware scheduling [17]. Consequently, in-memory computing integration not only alleviates the capacity limitations of single GPUs but also ensures sustained pipeline saturation, offering a scalable solution for serving large-scale models in resource-constrained environments [18].

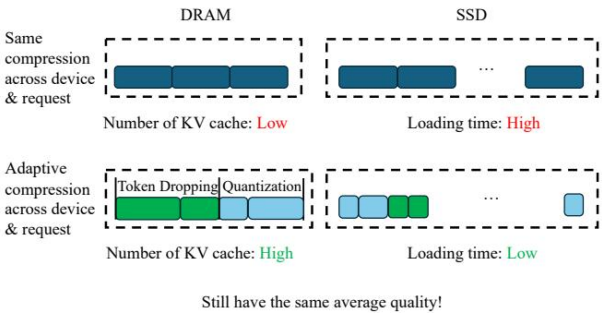

Figure 18: Chart from 'AdaptCache: KV Cache Native Storage Hierarchy for Low-Delay and High-Quality Language Model Serving' (arXiv:2509.00105)

2.4.2 Lookup Table Based Acceleration

Lookup Table (LUT) based acceleration has emerged as a critical paradigm for optimizing Large Language Model inference by fundamentally altering the dequantization workflow. Traditional arithmetic-based approaches often incur significant latency penalties when converting low-bit weights back to higher precision during matrix multiplication. In contrast, LUT-centric methodologies, such as LUT-GEMM, pre-compute possible dequantized values or partial products and

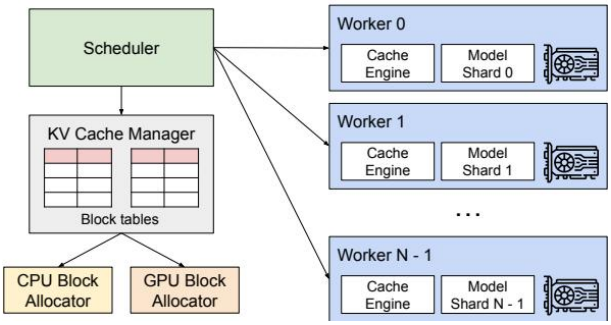

Figure 19: Chart from 'Efficient Memory Management for Large Language Model Serving with PagedAttention' (arXiv:2309.06180)

store them in on-chip memory. This strategy effectively replaces complex floating-point arithmetic operations with efficient memory access patterns, thereby minimizing the computational overhead associated with dynamic dequantization and enabling higher throughput for quantized models.

The architectural advantage of this approach lies in its ability to decouple computation intensity from memory bandwidth constraints through algorithm-hardware co-design. By treating table lookups as the primary computational primitive, these accelerators achieve superior performance compared to conventional integer arithmetic units, particularly in scenarios dominated by linear layers. The design typically involves partitioning weight matrices into small blocks, where each block's potential output combinations are indexed directly. This allows the hardware to sustain high utilization rates even under extreme quantization schemes, such as 4-bit or lower, by ensuring that the latency of fetching pre-computed results is substantially lower than executing the equivalent sequence of shift, add, and multiply instructions.

However, the efficacy of LUT-based acceleration is contingent upon careful management of memory resources and input dimensionality. As sequence lengths increase, the relative benefit may diminish if the attention mechanism's floating-point costs dominate the overall latency or if the lookup tables exceed available on-chip storage, forcing slower off-chip accesses. Consequently, advanced implementations employ hybrid strategies, utilizing LUTs for weight-dominated phases while optimizing attention computations separately. Furthermore, techniques like online-offline hybrid thresholding are often integrated to balance the trade-off between accuracy and the size of the required lookup structures, ensuring

that the acceleration remains robust across varying model scales and hardware configurations without compromising predictive fidelity.

2.4.3 Bit-Plane Disaggregation Schemes

Bit-plane disaggregation schemes represent a granular evolution of memory optimization, extending the concept of phase separation to the internal binary representation of tensors. Unlike traditional Prefill-Decode (PD) disaggregation which isolates compute-bound and memory-bound stages across distinct hardware units, bit-plane approaches decompose quantized weights and Key-Value (KV) caches into individual bit layers. This decomposition allows the system to treat significant bit-planes, which contain the majority of semantic information, differently from less significant planes that primarily contribute to fine-grained noise reduction. By isolating these planes, architectures can apply heterogeneous storage and transmission strategies, such as keeping high-order bits in fast on-chip SRAM while offloading lower-order bits to higher-latency DRAM or compressing them more aggressively without incurring substantial accuracy degradation.

The core mechanism leverages the observation that inference sensitivity varies non-linearly across bit positions. In this paradigm, the most significant bit-planes are processed with low-latency, high-bandwidth pathways to ensure immediate availability during the memory-bound decoding phase, effectively mitigating the bottleneck caused by full-tensor fetching. Conversely, less critical bit-planes can be retrieved asynchronously or reconstructed via lightweight approximation techniques when specific precision thresholds are not met. This selective access pattern significantly reduces the effective memory footprint and bandwidth requirements per token generation step. Furthermore, this scheme facilitates dynamic precision scaling, where the number of active bit-planes can be adjusted at runtime based on current workload demands or latency constraints, offering a flexible trade-off between inference speed and model fidelity.

Implementing bit-plane disaggregation requires specialized memory controllers and kernel modifications to handle non-contiguous bit-level data streams efficiently. Recent advancements integrate this approach with lookup table (LUT) based multiplication, where codebooks are similarly stratified by bit significance to optimize cache utilization. By aligning bit-plane retrieval with the hierarchical nature of modern memory systems, these schemes minimize data movement overheads that typically plague low-bitwidth quantization methods. Although current implementations often rely on custom hardware support for efficient bit-slicing and reassembly, software-emulated versions demonstrate promising throughput improvements in long-context scenarios. As LLMs scale, bit-plane disaggregation offers a viable path to sustain performance gains where traditional uniform quantization and coarse-grained model parallelism reach their diminishing returns.

3 System-Level KV Cache Management and Scheduling

3.1 Virtualized Memory Abstraction and Paging

3.1.1 Non-Contiguous Block Mapping

Non-contiguous block mapping has emerged as a fundamental strategy to mitigate GPU memory fragmentation and accommodate dynamic sequence lengths in large language model inference. By partitioning the Key-Value (KV) cache into fixed-size blocks allocated on demand, modern systems decouple logical token continuity from physical memory adjacency. This paged layout allows the memory manager to assign scattered physical pages to a single logical sequence, significantly improving memory utilization compared to traditional contiguous allocation schemes that suffer from internal fragmentation when handling variable-length requests [19].

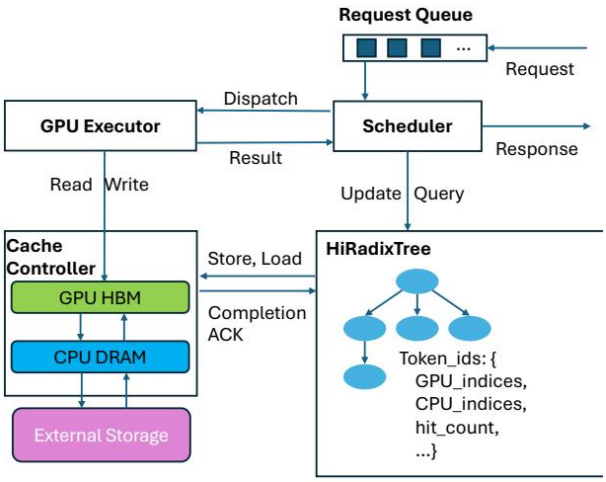

Figure 20: Chart from 'Strata: Hierarchical Context Caching for Long Context Language Model Serving' (arXiv:2508.18572)

However, this discontinuity introduces substantial overheads in data movement and metadata management. Unlike contiguous buffers that enable efficient bulk transfers via standard memory copy operations, non-contiguous mappings require complex scatter-gather mechanisms to orchestrate data movement between hierarchical storage tiers or across distributed nodes. The system must maintain intricate block tables that track the physical location of each logical block per layer, leading to increased pointer-chasing behaviors that map poorly to Single Instruction Multiple Thread (SIMT) execution models. Consequently, the latency associated with resolving these indirect memory accesses can become a bottleneck, particularly during the decoding phase where low-latency retrieval is critical [20].

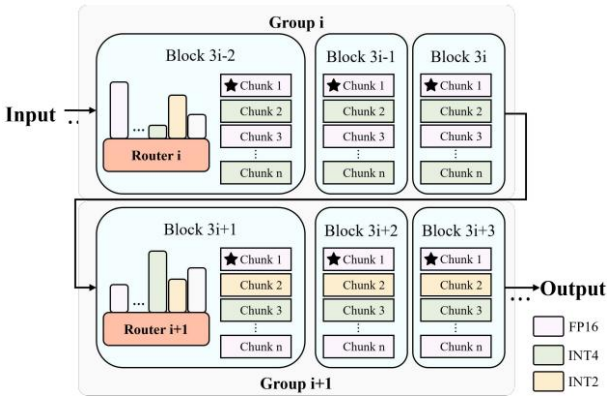

Figure 21: Chart from 'MoQAE: Mixed-Precision Quantization for Long-Context LLM Inference via Mixture of Quantization-Aware Experts' (arXiv:2506.07533)

To address these challenges, advanced implementations extend standard paging mechanisms with layer-wise residency tracking and dynamic block reallocation. Techniques such as block-level remapping and asynchronous stream execution are employed to overlap data transfers with computation, thereby hiding the latency inherent in non-contiguous access patterns. Furthermore, optimization strategies often involve aggregating sparse blocks into larger contiguous chunks for transfer operations or utilizing specialized hardware features to accelerate indirect addressing. Despite the metadata overhead and complexity in orchestration, non-contiguous block mapping remains the de facto standard for scalable inference, enabling systems to support massive context windows and high concurrency within constrained memory budgets.

3.1.2 Copy-On-Write Disaggregation

Copy-On-Write (COW) disaggregation fundamentally transforms KV cache management by decoupling logical state ownership from physical memory allocation through pointer-based indirection. Instead of duplicating heavy key-value tensors during context sharing or migration, systems maintain a unified memory pool where multiple requests reference identical physical blocks via lightweight metadata handles. This approach eliminates redundant data movement across network boundaries and GPU memory hierarchies, significantly reducing bandwidth consumption and latency overheads associated with traditional deep copying mechanisms in multi-tenant environments.

The implementation leverages zero-copy semantics to facilitate efficient cross-query reuse, particularly beneficial for prefix caching in Retrieval-Augmented Generation and multi-turn conversations. When a write operation targets a shared block, the system triggers a fault handler that allocates a new physical page and copies only the modified data, preserving the original immutable state for other readers. This granular isolation ensures that concurrent inference jobs can safely share common prompt prefixes without interference, while dynamic updates remain localized to specific request contexts, thereby maximizing memory utilization density without compromising data integrity.

Furthermore, COW disaggregation enables robust elasticity under dynamic workloads by allowing the scheduler to migrate logical states instantly through pointer remapping rather than bulk data transfer. This capability supports fine-grained resource balancing and rapid scaling of decoding instances, as the underlying storage layer handles versioning and consistency transparently. By minimizing the critical path overhead of state movement, this architecture sustains high throughput even when facing fluctuating demand patterns, establishing a foundational primitive for service-aware, disaggregated large language model serving infrastructures.

3.1.3 Hierarchical Address Space Management

Hierarchical address space management in modern LLM serving systems addresses the critical tension between high-bandwidth memory scarcity and the expansive storage requirements of long-context inference [21]. By decomposing the mem-

ory hierarchy into distinct tiers-typically ranging from on-chip SRAM and HBM to host DRAM and NVMe SSDs-these systems treat the address space as a unified, virtualized pool rather than disjoint physical regions. This abstraction enables fine-grained placement of KV cache blocks, where frequently accessed "hot" prefixes reside in low-latency tiers while colder context segments are offloaded to capacity-optimized storage without disrupting the logical continuity of the attention mechanism [19].

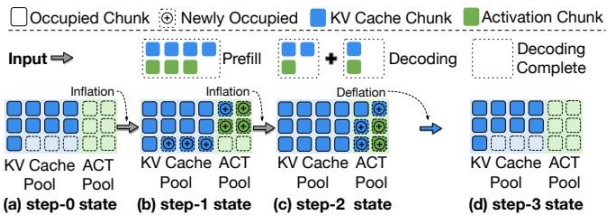

Figure 22: Chart from 'eLLM: Elastic Memory Management Framework for Efficient LLM Serving' (arXiv:2506.15155)

The core mechanism relies on block-aligned, page-oriented indexing that maps logical token sequences to physical locations across heterogeneous devices. Advanced implementations employ radix-tree structures or global prompt trees to identify shared prefixes, ensuring that identical logical chunks map to canonical physical pages regardless of their offset in the request context. This position-independent representation facilitates efficient deduplication and allows the runtime manager to dynamically migrate blocks between tiers based on real-time access frequency and reuse patterns. Consequently, the system minimizes redundant recomputation and mitigates bandwidth contention by aligning data movement with the specific granularity of the underlying storage backends.

Furthermore, hierarchical management integrates tightly with scheduling policies to optimize the trade-off between loading latency and recomputation costs. Runtime controllers utilize predictive models to estimate the time required for data transfer versus kernel execution, dynamically selecting the most cost-effective strategy for each block. By maintaining a global view of memory pressure and network topology, these managers can proactively prefetch anticipated context segments or evict stale blocks before they cause pipeline stalls. This adaptive approach ensures scalable concurrency under multi-tenant workloads, effectively decoupling the logical context length from the physical memory constraints of individual accelerators.

3.2 Disaggregated Serving and Cache Reuse

3.2.1 Prefix Matching via Radix Trees

Prefix matching via radix trees has emerged as a foundational mechanism for optimizing Key-Value (KV) cache reuse in large language model inference [22]. By organizing cached token sequences into a trie-based structure, systems can efficiently identify the longest common prefix (LCP) across diverse requests, transforming expensive prefill computations into lightweight memory lookups. This approach, exemplified by RadixAttention, enables fine-grained sharing of KV blocks among concurrent sessions, significantly reducing time-to-first-token latency and improving overall throughput without compromising generation quality.

$$\Delta H = H' - H \approx \frac{\partial H}{\partial K}\Delta K + \frac{\partial H}{\partial V}\Delta V$$

Figure 23: Chart from 'KVShare: An LLM Service System with Efficient and Effective Multi-Tenant KV Cache Reuse' (arXiv:2503.16525)

Advanced implementations extend the standard radix tree to support demand-aware retention and hierarchical storage tiers. Rather than functioning solely as a static index, modern variants annotate boundary nodes with metadata regarding access frequency or importance, guiding eviction policies under memory pressure. These structures often span heterogeneous memory pools, referencing data located in GPU VRAM, CPU DRAM, or SSDs through unified pointers. Such multi-tier designs allow the scheduler to maintain a global view of prompt distribution, facilitating intelligent dispatching that maximizes cache hit rates while minimizing data migration overhead across devices.

To further enhance scalability, recent innovations integrate segment-based indexing and dynamic pruning strategies within the radix framework. By inserting logical separators to define immutable chunks, systems can perform selective cross-prefix reuse independent of specific prompt placements, effectively decoupling chunk identity from sequence position. Additionally, bi-directional pruning techniques leverage compression-rate trade-offs to restrict the search

space, discarding infeasible candidates early during exploitation. These algorithmic refinements ensure that prefix matching remains computationally tractable even as context lengths expand and concurrency levels increase in production environments [15].

3.2.2 Cross-Request Cache Sharing Protocols

Cross-request cache sharing protocols fundamentally shift LLM serving from static, request-isolated memory management to dynamic, service-aware resource pooling. Traditional approaches often employ monolithic offloading policies that ignore batch-dimension mismatches, leading to suboptimal placement and avoidable computational stalls. Modern systems address this by implementing fine-grained, per-request placement strategies that leverage the deterministic nature of layer-wise computation. By decoupling KV cache storage from specific GPU layers and utilizing global memory pools, these protocols enable efficient overlap between data transfer and computation, significantly reducing Time To First Token (TTFT) under high concurrency [23].

$$blocks\_to\_fetch = \sum_{r \in \mathcal{R}} \sum_{\ell=1}^{L} b_r \, (1 - x_{r,\ell}).$$

Figure 24: Chart from 'OrbitFlow: SLO-Aware Long-Context LLM Serving with Fine-Grained KV Cache Reconfiguration' (arXiv:2601.10729)

A critical evolution in this domain is the transition from prefix-only caching to position-independent and segment-based sharing mechanisms [24]. While early methods relied on strict prefix matching, advanced protocols now utilize content-segment hashing to identify and reuse common KV blocks across requests with divergent private histories. This capability is further enhanced by prefix-locality-aware routing policies, which pin requests sharing common contexts to consistent prefill workers to maximize cache hit rates. Such techniques allow systems to recover sharing beyond simple prefixes, mitigating redundant recomputation costs even when absolute token positions differ among concurrent sessions.

Furthermore, robust cross-request sharing necessitates adaptive scheduling algorithms that co-design admission control with memory management to handle bandwidth contention and SLO requirements. Systems increasingly integrate predictive uploading and proactive offloading to balance memory footprint against latency constraints, dynamically switching between token-level and KV-level migration based on real-time workload characteristics. By restricting solution spaces to disjoint server chains or employing two-time-scale algorithms for block placement, these protocols ensure stable performance in multi-tenant environments. Ultimately, the synergy between intelligent request scheduling and flexible cache resizing transforms static compression methods into practical, high-throughput solutions for disaggregated inference.

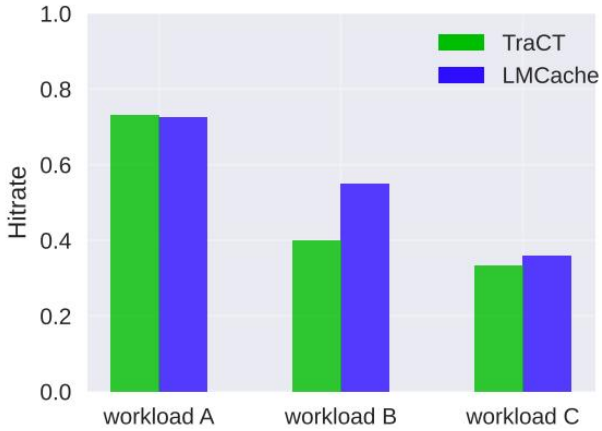

Figure 25: Chart from 'TraCT: Disaggregated LLM Serving with CXL Shared Memory KV Cache at Rack-Scale' (arXiv:2512.18194)

3.2.3 Distributed Object Storage Integration

Distributed object storage integration has emerged as a critical architectural component for scaling Key-Value (KV) cache management beyond the constraints of local GPU High Bandwidth Memory. By decoupling compute and storage resources, modern systems leverage disaggregated pools to accommodate massive context windows and facilitate cross-instance cache sharing. This paradigm shifts the KV cache from a transient, request-scoped intermediate to a persistent, globally addressable asset, enabling techniques such as live migration and prefix reuse across diverse inference nodes without incurring the prohibitive costs of recomputation or rigid memory partitioning [25].

To address the latency disparities inherent in remote access, advanced frameworks employ hierarchical caching strategies that intelligently tier data between fast local memory and slower distributed

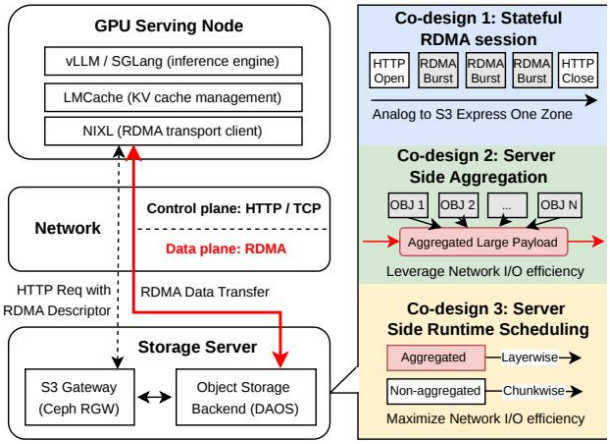

Figure 26: Chart from 'ObjectCache: Layer-wise Object-Storage Retrieval for KV Cache Reuse' (arXiv:2605.22850)

object stores. These systems utilize cost estimators to dynamically decide whether to retain, offload, or recompute specific cache blocks based on access frequency, context length, and storage bandwidth characteristics. Furthermore, optimized serialization formats, including block-sparse differential encoding and transform coding, significantly reduce the footprint of transferred data, allowing high-throughput retrieval of redundant context segments while minimizing network congestion during peak concurrency.

Despite these advancements, maintaining consistency and low latency in a distributed environment remains challenging due to network heterogeneity and the complexity of coordinating stateful operations across disjoint hardware. Current solutions mitigate these issues through sophisticated scheduling algorithms that overlap data transfer with computation and employ predictive prefetching mechanisms to hide retrieval latency. However, the fundamental tension between storage capacity and access speed necessitates continued innovation in protocol design, particularly for scenarios requiring strict tail-latency guarantees in multi-tenant, large-scale deployment environments where cache locality cannot be assumed.

3.3 Dynamic Scheduling and Resource Orchestration

3.3.1 Service Level Objective Aware Batching

Service Level Objective (SLO) aware batching transforms request aggregation from a static throughput maximization task into a dynamic, constraint-driven optimization problem. Unlike traditional continuous batching that groups requests based solely on arrival times or sequence lengths, SLO-aware mechanisms incorporate strict latency budgets, such as Time to First Token (TTFT) and inter-token latency, directly into the scheduling decision loop. This approach recognizes that the latency-optimal configuration is not fixed but varies significantly across different workloads, bandwidth regimes, and service states. Consequently, the system must continuously evaluate whether a candidate batch composition satisfies the aggregate SLO requirements before committing resources, treating KV cache compression and offloading strategies as variables dependent on the current service context rather than predetermined algorithms.

To enforce these constraints, advanced schedulers implement fine-grained, per-request control mechanisms that monitor token-level violation rates against tunable upper bounds. When a proposed batch configuration threatens to exceed SLO caps, the solver identifies the placement as infeasible and triggers fallback protocols, such as batch trimming or request re-ordering, to restore compliance. This often involves disaggregating prompt processing from token generation to mitigate pipeline bubbles, ensuring that memory footprints for active microbatches fit within available GPU capacity while maximizing throughput. By maintaining a global view of request priorities and predicted execution times, the scheduler can dynamically adjust batch sizes and offload distances, preventing scenarios where a single long-running request stalls critical-path agents or causes cascading failures across in-flight microbatches.

The integration of SLO awareness fundamentally alters the trade-off between system throughput and tail latency, particularly under high-load conditions with variable context lengths. While naive batching strategies suffer from performance degradation as batch sizes increase due to amplified drifts in both token and batch dimensions, SLO-aware algorithms sustain efficiency by adaptively selecting optimal colocations and compression ratios. This results in significant reductions in end-to-end latency and kernel launch overheads, enabling the system to handle diverse workloads ranging from interactive chatbots to long-form generation without violating strict service guarantees. Ultimately, this paradigm shifts the focus from maximizing raw hardware utilization to optimizing user-perceived performance through intel-

ligent, real-time adaptation to fluctuating demand patterns.

3.3.2 Interference-Free Request Isolation

Interference-free request isolation primarily targets the decoupling of compute-intensive prefill and latency-sensitive decode phases to eliminate resource contention. By physically separating these stages onto dedicated GPU pools, systems prevent incoming large-context prompts from stalling autoregressive generation, thereby stabilizing token-level latency. This architectural shift enables independent scaling of heterogeneous hardware, ensuring that bursty arrival patterns in the prefill stage do not propagate queuing delays to the decoding pipeline, which is critical for maintaining strict Service Level Objectives under high concurrency.

Beyond phase separation, advanced isolation mechanisms employ fine-grained per-request placement strategies to mitigate bandwidth saturation and memory conflicts. Unlike uniform offloading policies that ignore batch dimension mismatches, modern schedulers dynamically allocate KV cache blocks and communication bandwidth based on real-time runtime signals. This approach allows the system to overlap computation with data transfer effectively, proactively pausing specific requests to free resources for others, thus achieving a global placement that minimizes end-to-end response time while avoiding the stalls associated with rigid, static configurations.

Furthermore, robust isolation requires explicit management of cross-request dependencies to prevent interference during shared resource access. Techniques such as logical block delimitation and round-aware prompt interfaces enable the runtime to distinguish between private and shared context segments, facilitating efficient KV cache reuse without compromising generation quality [26]. By enforcing strict boundaries on memory access and utilizing predictive upload mechanisms, these systems ensure that concurrent requests operate in near-isolation, significantly reducing tail latency and preventing the performance degradation typically caused by noisy neighbors in multi-tenant serving environments.

3.3.3 Predictive Length Estimation Models

Predictive length estimation models address the inherent stochasticity of autoregressive generation, where prompt length correlates poorly with response magnitude. Traditional approaches frame this challenge as a multi-class classification problem, discretizing the continuous output space into percentile ranges to predict the likely bucket for a given sequence. This granularity mitigates the high variance observed in raw token counts, enabling schedulers to anticipate resource demands with greater stability than direct regression methods, which often suffer from significant error margins in long-tail distributions.

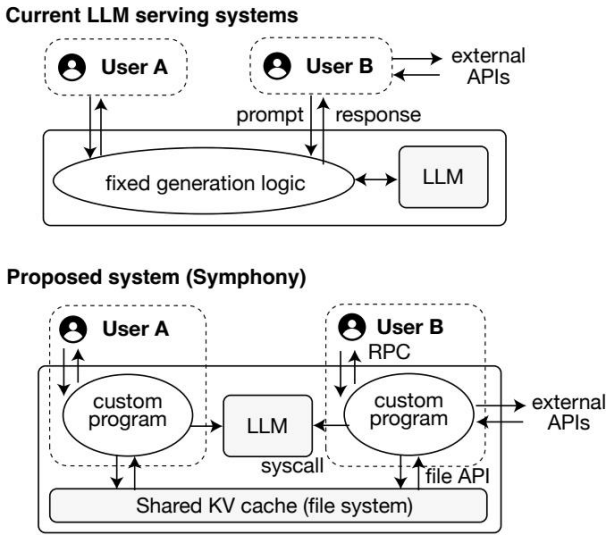

Figure 27: Chart from 'Serve Programs, Not Prompts' (arXiv:2510.25412)

To enhance accuracy, modern frameworks employ hybrid forecasting strategies that combine cold-start static analysis with dynamic runtime adaptation. Initially, pre-runtime profiling provides a baseline estimate derived from model architecture and historical data patterns. Upon the first execution of a request type, the system transitions to an adaptive exponentially weighted moving average model, continuously refining predictions by integrating real-time feedback loops. This dual-phase approach allows the estimator to correct for policy-sensitive variations, such as those induced by different decoding strategies or complex agentic workflows, ensuring robust performance across diverse workload characteristics.

The integration of these predictive mechanisms is critical for optimizing memory management and scheduling policies in distributed inference environments. Accurate length forecasts facilitate proactive Key-Value cache allocation, reducing fragmentation and minimizing the overhead associated with dynamic memory resizing. Furthermore, these estimates drive intelligent offload-

ing decisions by comparing predicted computation durations against data transfer latencies, effectively balancing compute-bound and memory-bound constraints. By leveraging proxy models or lightweight auxiliary networks to perform these estimations, systems achieve near-optimal resource utilization without incurring prohibitive computational costs during the inference path.

3.4 Heterogeneous Memory Tiering and Offloading

3.4.1 GPU-CPU-SSD Hierarchical Swapping

GPU-CPU-SSD hierarchical swapping addresses the critical memory capacity bottleneck in large language model serving by extending the memory hierarchy beyond high-bandwidth GPU HBM. As context lengths expand, the KV cache frequently exceeds available device memory, necessitating the offloading of inactive blocks to slower CPU DRAM and persistent NVMe SSD storage [19]. This tiered architecture treats the KV cache as a movable I/O payload, dynamically migrating data across devices based on access frequency and request priority to maximize the effective batch size without compromising model fidelity [27].

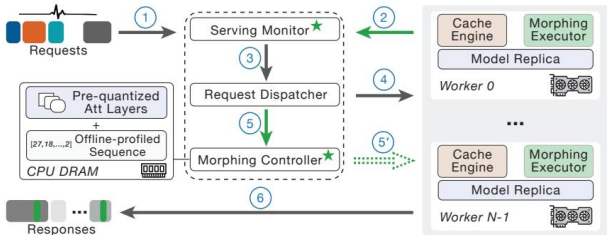

Figure 28: Chart from 'Efficient and Workload-Aware LLM Serving via Runtime Layer Swapping and KV Cache Resizing' (arXiv:2506.02006)

The implementation of this hierarchy relies on sophisticated asynchronous data movement strategies to conceal latency inherent in PCIe and NVMe interfaces. Systems employ dedicated CUDA streams and pinned host memory to overlap data transfers with ongoing compute operations, ensuring that GPU execution units remain saturated [18]. Advanced mechanisms, such as diff-aware storage and fused restore paths, further optimize bandwidth utilization by transmitting only sparse differences against shared master copies rather than full cache blocks, while ping-pong buffering techniques enable continuous ingestion and correction of data during layer-wise processing.

However, managing this multi-tiered environment introduces complex scheduling challenges regarding placement granularity and prefetching accuracy. Static allocation strategies often fail to adapt to dynamic token generation rates, leading to severe SLO violations when data retrieval stalls the decoding pipeline. Consequently, modern approaches utilize online solvers and predictive models to estimate execution times and data-transfer overheads, dynamically adjusting the residency of KV blocks across GPU, CPU, and SSD layers. This coordinated management ensures that the latency penalty of accessing slower storage tiers is minimized while maintaining high throughput across heterogeneous hardware pools.

3.4.2 Bandwidth-Optimized Data Transfer

Bandwidth-optimized data transfer strategies critically depend on the underlying network regime, as communication savings from compression or offloading diminish once bandwidth exceeds specific thresholds. Empirical profiling across consumer, workstation, and data-center tiers reveals that static compression policies often degrade latency when interconnect speeds surpass 50 to 110 Gbps, rendering decompression overheads counterproductive. Consequently, effective systems must treat bandwidth as a dynamic service state, adapting transfer mechanisms to prevent communication from becoming the dominant bottleneck, particularly in Ethernet-connected cloud environments where KV cache migration costs can severely amplify job completion times.

To mitigate these constraints, advanced frameworks employ fine-grained, per-request placement policies that overlap data transfer with computation, effectively hiding latency during decoding phases. Techniques such as fused retrieval and diff-aware storage encode cache differences against shared master copies, reducing both compute work and network traffic by avoiding separate reconstruction steps. Furthermore, co-optimizing memory layouts with network primitives, such as utilizing huge pages and buffered copies, addresses inefficiencies arising from naive discrete memory structures, achieving significant throughput improvements compared to baseline contiguous transfers.

Disaggregated serving architectures further enhance bandwidth utilization by decoupling prefill and decoding stages onto dedicated resources, ensuring that high-bandwidth interconnects are

not saturated by mixed traffic patterns. By dynamically selecting the cheapest storage device where loading delay remains below recomputation time, systems can navigate trade-offs between network bandwidth and local compute capacity. This adaptive approach, combined with predictive upload mechanisms and optimized PCIe transfer pipelines, ensures that parameter swapping and KV cache migration occur with negligible overhead, sustaining low tail latency even under varying request arrival rates and context lengths.

3.4.3 Compression-Aware Eviction Algorithms

Compression-aware eviction algorithms address the limitations of static strategies by dynamically adapting to fluctuating network conditions and growing cache footprints. Unlike traditional methods that apply fixed heuristics, these approaches recognize that an optimal placement early in decoding often becomes suboptimal or infeasible as sequences extend [28]. Consequently, modern systems employ dynamic reconfiguration mechanisms that treat compression not as a isolated step, but as a composable component within a unified pipeline. This integration allows for the simultaneous optimization of compression ratios and accuracy, mitigating the performance degradation often observed when rotation, quantization, and entropy coding are studied in isolation.

$$x_i' = \arg\max p_i,$$

Figure 29: Chart from 'A Comprehensive Survey of Accelerated Generation Techniques in Large Language Models' (arXiv:2405.13019)

To navigate the complex trade-offs between memory footprint and inference fidelity, advanced algorithms utilize demand-aware policies that prioritize critical tokens while discarding less informative data. Techniques such as adaptive pruning leverage the persistence of importance hypothesis to identify evictable candidates, often employing min-heap structures keyed by specialized eviction metrics. These methods frequently incorporate protection budgets to safeguard system-critical nodes and reusable anchors, ensuring that aggressive compression does not compromise the coherence of the attention mechanism. Furthermore, by distinguishing between active and inactive layers, these algorithms can selectively remap parameters based on circular execution characteristics, theoretically guaranteeing optimality under specific constraints.

The implementation of these strategies often involves sophisticated block placement and cache reservation schemes to minimize data transfer stalls and maximize hardware utilization. Greedy algorithms sequentially allocate blocks to form efficient server chains, balancing service rates against strict latency service-level objectives. By integrating lazy eviction policies with remote persistence layers, systems can effectively handle bursty memory allocation requests without incurring prohibitive latency penalties. This holistic approach enables the navigation of the accuracy-latency Pareto frontier, allowing serving instances to adopt larger batch sizes and sustain higher throughput even under resource-constrained environments, ultimately outperforming rigid, heuristic-based baselines across diverse workloads.

4 Speculative Decoding for Accelerated Inference

4.1 Draft Model Architectures and Selection

4.1.1 Heterogeneous Small Language Model Chains

Heterogeneous Small Language Model (SLM) chains represent a strategic architectural shift designed to mitigate the latency bottlenecks inherent in autoregressive Large Language Model (LLM) inference [29]. By orchestrating a sequence of models with varying parameter counts and aspect ratios, these systems leverage the observation that token prediction difficulty fluctuates throughout generation. Instead of relying on a monolithic model for every step, heterogeneous chains deploy lightweight drafters, such as OPT-125M, to rapidly propose token sequences for verification by larger targets like OPT-6.7B. This approach capitalizes on the robustness of language model performance to changes in model aspect ratio, allowing for the deployment of wider, shallower draft models that minimize communication overhead while maintaining high acceptance rates during the verification phase.

The efficacy of these chains relies heavily on the complementary strengths of the constituent models, where smaller units capture short-range de-

$$\mathcal{L}_{\text{GC}} = \mathbb{E}_{(\boldsymbol{x},\mathcal{J})\sim\mathcal{D},\boldsymbol{y}\sim\mathcal{J}}\Big[\sum_{i=1}^{n} D\left(q_{\theta^-}(\cdot|\boldsymbol{y}^*_{<i},\boldsymbol{x})||q_\theta(\cdot|\boldsymbol{y}_{<i},\boldsymbol{x})\right)\Big]$$

Figure 30: Chart from 'CLLMs: Consistency Large Language Models' (arXiv:2403.00835)

pendencies efficiently while larger models resolve complex, long-range contextual nuances. Research indicates that lower layers of pretrained models primarily focus on local patterns, suggesting that shallow SLMs can serve as effective proxies for initial token generation without sacrificing overall coherence. By training wider models with fewer layers specifically for the drafting role, systems can achieve rapid sampling on the same hardware as the target model. This configuration reduces the computational cost per token during the speculative phase, ensuring that the wall-clock time saved through parallel verification outweighs the overhead of running multiple distinct model instances.

Furthermore, heterogeneous chains offer significant engineering advantages by enabling dynamic resource allocation based on task complexity. Unlike homogeneous ensembles, which often incur redundant computational costs, heterogeneous setups allow for the integration of specialized models optimized for specific linguistic domains or dependency ranges. This modularity facilitates the construction of robust inference pipelines where easy tokens are resolved by minimal compute resources, reserving the full capacity of the large target model only for ambiguous or high-entropy predictions. Consequently, this paradigm not only accelerates inference throughput but also enhances scalability in resource-constrained environments, providing a flexible framework for deploying advanced language capabilities without the prohibitive latency of traditional sequential decoding.

4.1.2 Self-Speculative Internal Mechanisms

Self-speculative internal mechanisms represent a paradigm shift from traditional speculative decoding by eliminating the requirement for a separate, lightweight draft model [30]. Instead, these approaches leverage the target large language model itself to generate candidate tokens, thereby reducing memory overhead and simplifying deployment architectures. The core insight driving this methodology is that skipping specific intermediate layers or utilizing early-exit strategies within the same network can produce sufficiently accurate draft sequences without significantly compromising the final output distribution. This self-contained approach allows the model to act simultaneously as both the proposer and verifier, capitalizing on the redundancy often present in deep transformer stacks to accelerate inference.

Technically, these mechanisms employ strategies such as layer skipping, where the model bypasses computationally expensive attention or feed-forward layers during the drafting phase to rapidly generate multiple future tokens. The verification stage then involves a full forward pass through the complete network to validate these candidates using standard rejection sampling protocols. Advanced implementations incorporate adaptive thresholds based on token-level uncertainty or confidence scores, enabling the system to dynamically regulate its drafting aggressiveness. By comparing the hidden states or logits from the shallow draft pass against the deep verification pass, the algorithm ensures that only tokens adhering to the target model's probability distribution are accepted, maintaining theoretical guarantees of lossless generation.

Despite the architectural elegance of self-speculation, challenges remain regarding the trade-off between drafting speed and acceptance rates. Unlike distinct draft models that are explicitly trained for distribution alignment, internal mechanisms must rely on the inherent capabilities of the truncated network, which can lead to higher rejection rates if the skipped layers contain critical contextual information. Recent advancements address this by integrating hierarchical verification schemes and fine-tuning early-exit policies to minimize error propagation. Furthermore, these methods offer significant advantages in resource-constrained environments by maximizing hardware utilization through parallel execution of drafting and verification within a single model instance, effectively mitigating the memory bottlenecks associated with loading multiple models.

4.1.3 Retrieval-Based Token Prediction

Retrieval-based token prediction diverges from parametric draft models by leveraging non-parametric datastores to speculate future sequences. Instead of training a separate lightweight

network, these methods utilize historical token contexts as queries to retrieve matching subsequences from a pre-constructed index, such as a Trie or vector store. The retrieved continuations serve as candidate tokens, effectively capitalizing on the redundancy inherent in natural language and code corpora. This approach allows for the integration of speculative decoding with arbitrary large language models without requiring additional fine-tuning or architectural modifications [30].

The verification mechanism in retrieval-based systems must address the variable length and branching nature of retrieved candidates. Unlike fixed-length drafts, retrieved sequences often form a token tree rather than a linear chain, necessitating specialized verification strategies like token-tree attention to evaluate multiple paths simultaneously [31]. The system validates these candidates against the target model's distribution, accepting tokens only where the retrieval aligns with the model's probabilistic output. If a mismatch occurs at any index, the speculative chain terminates, and the target model resamples the divergent token, ensuring the final output distribution remains identical to standard autoregressive decoding.

$$\mathbf{X}_{\mathrm{INT}} = \left[\frac{\mathbf{X}_{\mathrm{FP16}} - Z}{S} \right],$$

Figure 31: Chart from 'A Survey on Efficient Inference for Large Language Models' (arXiv:2404.14294)

Despite its efficiency gains, this paradigm faces challenges regarding retrieval latency and coverage sparsity. The effectiveness of speculation is contingent upon the density of the datastore; rare tokens or novel contexts may yield no matches, forcing a fallback to standard sequential generation. Furthermore, dynamic tree decoding strategies are often employed to optimize the trade-off between retrieval overhead and acceptance rates, adapting the depth of speculation based on prompt complexity. Recent advancements focus on optimizing the indexing structure and attention computation to minimize the latency introduced by the retrieval process, ensuring that the speedup from parallel verification outweighs the cost of datastore queries.

4.2 Tree-Structured Parallel Verification

4.2.1 Adaptive Draft Tree Construction

Adaptive draft tree construction transcends static, manually designed topologies by dynamically optimizing the branching factor and depth based on real-time inference conditions. Unlike fixed linear chains or pre-defined trees, modern approaches leverage confidence scores, entropy metrics, and hardware constraints to modulate the tree structure at each decoding step. This adaptability ensures that the system allocates computational resources efficiently, expanding the tree breadth-wise when the draft model exhibits high certainty and restricting growth during ambiguous generation phases to minimize verification overhead.

The core mechanism involves a hierarchical validation process where the target model verifies multiple candidate sequences in parallel using specialized tree attention masks. By sharing common prefixes across divergent paths, these methods maximize the number of leaf nodes relative to the number of forward passes required by the draft model. Advanced strategies further refine this by employing classifiers to predict optimal speculation lengths or by recursively integrating smaller auxiliary models to review generations, thereby increasing the probability of accepting longer contiguous token sequences while maintaining the theoretical guarantee of sampling from the target distribution.

Recent innovations have introduced hardware-aware optimizers and dynamic thresholding to further enhance this process. These systems continuously monitor acceptance rates and adjust the draft-exiting mechanisms or tree topology to balance memory bandwidth usage with computational throughput. By dynamically selecting high-quality subsets of drafts and organizing them into optimized tree structures, these adaptive methods significantly reduce rollback frequency and latency, offering robust performance improvements across varying sequence complexities and model sizes without compromising generation quality.

4.2.2 Multi-Candidate Attention Masking

Multi-candidate attention masking addresses the computational inefficiency of verifying multiple speculative token sequences by restructuring the attention mechanism to operate on tree-structured batches rather than flattened linear sequences [32]. In standard speculative decoding, verifying dis-

tinct candidate paths often requires redundant computation for shared prefixes [33]. By organizing draft tokens into a tree where nodes represent tokens and edges denote sequential dependencies, the system can process all candidates in a single forward pass. This approach necessitates a specialized attention mask that strictly enforces causal constraints within the tree topology, ensuring each token attends only to its valid ancestors while ignoring unrelated branches.

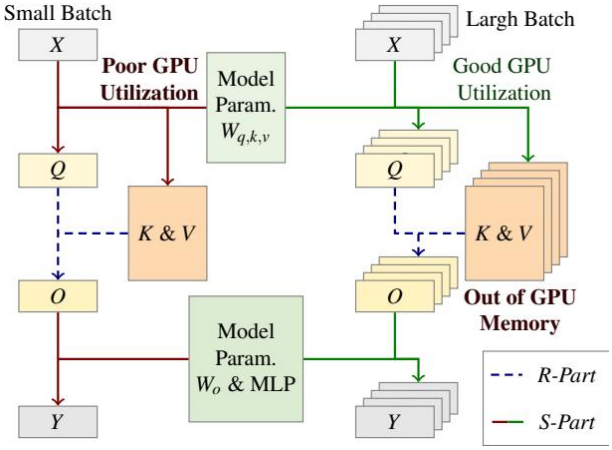

Figure 32: Chart from 'FASTDECODE: High-Throughput GPU-Efficient LLM Serving using Heterogeneous Pipelines' (arXiv:2403.11421)

$$\hat{\mathbf{X}}_{\text{INT8}} = Q(\mathbf{X} \odot \mathbf{s}, \Delta_x)$$

Figure 33: Chart from 'Quasar: Quantized Self-Speculative Acceleration for Rapid Inference via Memory-Efficient Verification' (arXiv:2603.01399)

The core technical innovation lies in the dynamic construction of the attention matrix to accommodate variable-depth trees and shared contexts. Unlike traditional causal masks that block all future positions, tree attention masks allow tokens at the same depth but different branches to remain mutually invisible, while permitting access to the common root path. This deduplication of shared prefixes significantly reduces the total number of tokens submitted to the target model for verification. Consequently, the attention computation scales with the number of unique nodes in the token tree rather than the sum of lengths of all candidate sequences, optimizing memory bandwidth usage and reducing latency during the verification phase [34].

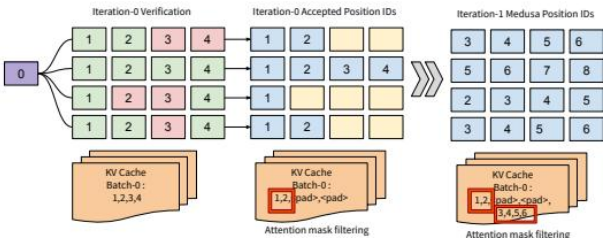

Figure 34: Chart from 'SpecMemo: Speculative Decoding is in Your Pocket' (arXiv:2506.01986)

Implementation of this mechanism requires careful modification of the key-value cache management and positional embedding strategies to align with the non-linear tree structure. The attention scores are computed such that the verification step effectively evaluates the joint probability of the entire candidate set in parallel. By integrating this masking strategy with high-performance attention kernels, systems can achieve substantial throughput gains, particularly when the speculation width is large. This method transforms the verification bottleneck from a sequential constraint into a parallelizable operation, enabling more aggressive speculation strategies without proportional increases in computational cost.

4.2.3 Optimal Transport Coupling Strategies

Optimal Transport (OT) coupling strategies provide a rigorous mathematical framework for aligning the probability distributions of draft and target models in speculative decoding [35]. By defining a cost function, typically the indicator function for token mismatch, the objective becomes finding a coupling plan that minimizes the expected transportation cost, which directly corresponds to the rejection rate. Unlike standard rejection sampling which operates sequentially, OT formulates the alignment as a global optimization problem over the joint distribution space, ensuring that the marginal constraints of both the draft and target distributions are strictly satisfied while maximizing the probability of token agreement.

$$\sigma(s_i^{(l)}) = \prod_{t=1}^{l} p_\theta(s_i^{(l)}[t] \mid s_i^{(l)}[1:t-1])$$

Figure 35: Chart from 'CARD: Cache-Assisted Parallel Speculative Decoding for Efficient Large Language Model Inference' (arXiv:2508.04462)

The implementation of these strategies often involves solving for the optimal coupling matrix that dictates how mass is transferred from the draft distribution to the target distribution. In practice, this allows for the construction of a modified sampling scheme where tokens are selected not merely based on the draft model's confidence, but according to a transport plan that maximizes the likelihood of acceptance by the larger target model. This approach theoretically guarantees that the output sequence remains unbiased with respect to the target model's true distribution, provided the transport cost is minimized effectively, thereby preserving generation quality while reducing the variance in acceptance lengths.

Recent advancements have integrated OT coupling with adaptive mechanisms to handle dynamic inference scenarios where computational budgets and latency constraints vary. By analyzing the derivative of the execution time function with respect to the speculation length, systems can identify an optimal operating point where the marginal gain in accepted tokens balances the overhead of additional forward passes. These adaptive strategies leverage the flexibility of OT plans to adjust coupling intensities in real-time, offering superior acceleration compared to static heuristic methods, particularly in regimes where the divergence between draft and target distributions is significant or when hardware memory bandwidth imposes strict limits on parallel verification.

4.3 Dynamic Speculation Control and Optimization

4.3.1 Context-Aware Speculation Length Adjustment

Static speculation lengths often fail to accommodate the dynamic nature of linguistic uncertainty, leading to suboptimal latency across varying traffic patterns. Empirical analysis reveals that while shorter speculation lengths, such as two tokens, excel during periods of dense request traffic, longer sequences of four tokens yield superior throughput when traffic is sparse. This variability stems from the inherent trade-off between drafting overhead and verification efficiency; a fixed parameter cannot simultaneously minimize computational waste in low-confidence contexts and maximize parallelism in high-certainty scenarios. Consequently, rigid configurations frequently underperform compared to an oracle that selects the optimal static length for each specific temporal window.

To mitigate these limitations, context-aware adaptive mechanisms dynamically adjust the speculation lookahead based on real-time system states and model confidence estimates. Rather than relying on exhaustive grid searches which are computationally prohibitive, these approaches employ lightweight classifiers or information-theoretic frameworks to determine whether the draft model should continue generating or transition immediately to target model verification. By monitoring factors such as batch size fluctuations and token acceptance rates, adaptive algorithms can modulate the drafting depth iteratively. This flexibility allows the system to expand the speculation window when the draft model demonstrates high alignment with the target, while contracting it during ambiguous generation phases to avoid unnecessary verification costs.

The integration of adaptive speculation length adjustment consistently matches or exceeds the performance of the best static baselines across diverse operational conditions. Experimental results indicate that such dynamic strategies reduce end-to-end latency by approximately 9% to 14% compared to fixed lengths of two and four tokens, respectively. These gains are particularly pronounced in dynamic traffic scenarios where request sparsity fluctuates, demonstrating that adaptability is crucial for maintaining optimal inference speeds. Ultimately, context-aware adjustment transforms speculative decoding from a static heuristic into a responsive optimization loop, effectively harnessing available computational resources to minimize idle time and maximize token acceptance efficiency without compromising output distribution fidelity.

4.3.2 Uncertainty-Modulated Acceptance Thresholds

Uncertainty-modulated acceptance thresholds address the limitations of static verification criteria in speculative decoding by dynamically adjusting the likelihood margin based on the drafter model's confidence [36]. Traditional rejection sampling employs a fixed threshold, which fails to account for the varying predictability of tokens across different contexts; easy tokens may be overly constrained, while difficult ones suffer from high rejection rates. To mitigate this, recent approaches introduce a confidence-sensitive verification gate where the acceptance threshold τ_t at token posi-

tion t is defined as a function of the model's uncertainty, typically calculated as $\tau_t = \tau_{base} + \gamma(1 - C_t)$. Here, C_t represents the confidence score derived from the drafter's output distribution, and γ serves as a tunable hyperparameter controlling the sensitivity of the threshold to uncertainty fluctuations.

This adaptive mechanism creates a feedback-controlled decoding loop that balances generation speed with output fidelity. When the drafter exhibits high confidence, the threshold lowers, allowing for a more aggressive speculation strategy that accepts longer sequences of drafted tokens. Conversely, as uncertainty increases, the threshold rises, enforcing stricter verification to prevent the propagation of errors that would necessitate costly rollbacks. This soft, interpretable regulation complements dynamic drafting windows, ensuring that the system speculates more tokens only when the probability of acceptance justifies the computational overhead. Empirical observations suggest that such dynamic adjustment yields more robust performance across diverse domains compared to rigid criteria, particularly in scenarios where token predictability varies significantly within a single sequence.

Furthermore, advanced implementations extend this concept by employing sequences of descending thresholds or entropy-dependent criteria to refine the selection of plausible candidates. Instead of a binary accept-reject decision based on a single value, these methods evaluate drafted tokens against a hierarchy of thresholds, terminating the search once a candidate satisfies the confidence requirement. This approach effectively relaxes acceptance criteria for low-entropy predictions while maintaining strict adherence to the target distribution for ambiguous tokens. By aligning the verification strictness with the inherent uncertainty of the drafting process, uncertainty-modulated thresholds optimize the trade-off between acceptance rate and computational efficiency, ultimately enhancing the overall throughput of large language model inference without compromising generation quality.

4.3.3 Early Exiting and Stopping Criteria

Early exiting mechanisms enable adaptive computation by allowing tokens to bypass deeper network layers once a confidence threshold is met, effectively reducing the average inference depth per token. This approach, extensively explored in encoder architectures, relies on hand-crafted patience or confidence criteria to determine the optimal exit layer dynamically. While bypassing layers significantly lowers floating-point operations by skipping KV cache updates for subsequent stages, it introduces a trade-off between latency and generation quality due to potential train-test mismatches in early-layer representations.

In the context of speculative decoding, stopping criteria govern the length of draft sequences generated before verification, balancing speculation gains against rejection penalties [37]. Static speculation lengths often fail to adapt to varying input complexities, whereas dynamic strategies utilize entropy-based lower bounds or moving averages of acceptance rates to adjust drafting windows in real-time. Advanced formulations derive theoretical relationships between draft model entropy and expected acceptance probabilities, enabling the system to halt drafting when the likelihood of token rejection outweighs the potential speedup from parallel generation.

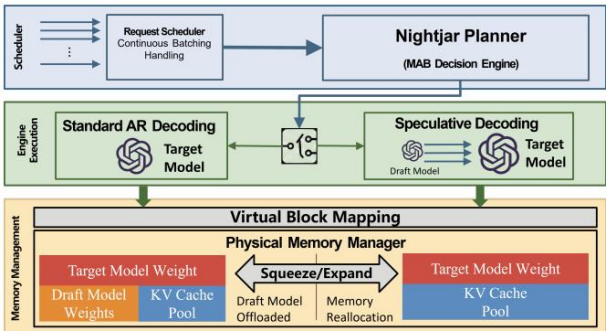

Figure 36: Chart from 'Nightjar: Dynamic Adaptive Speculative Decoding for Large Language Models Serving' (arXiv:2512.22420)

Sophisticated control policies further refine these decisions through fallback phases and deterministic rollback mechanisms triggered when draft confidence drops below specific thresholds. By monitoring the decay in prediction quality across reasoning steps, these methods prevent the wasteful generation of long, low-probability token chains that are likely to be discarded during verification. Consequently, integrating adaptive early exiting with intelligent stopping criteria allows systems to strike an optimal balance between inference throughput and model accuracy, ensuring efficient resource utilization across diverse prompt complexities without compromising output fidelity.

4.4 Distributed and Collaborative Speculation

4.4.1 Multi-Device Parallel Drafting

Multi-Device Parallel Drafting extends speculative decoding by distributing the generation of draft tokens across heterogeneous hardware resources to maximize throughput [28]. Unlike single-device approaches that rely on smaller surrogate models or auxiliary heads, this paradigm leverages multiple accelerators to execute the drafting and verification phases concurrently. By decoupling the computational graphs of the draft and target models onto separate devices, the system mitigates memory bandwidth bottlenecks and allows the large language model to focus exclusively on verification while the draft model predicts subsequent token sequences in parallel.

However, realizing effective speedups requires careful management of inter-device communication overheads, which can negate the benefits of parallelism, particularly for smaller model variants. The latency introduced by transferring intermediate activations or KV caches between devices must remain strictly lower than the time saved through concurrent execution. Consequently, optimal topology selection is critical; deploying a small draft model across an excessive number of devices often results in diminished returns due to synchronization delays, suggesting that hardware allocation must be dynamically adjusted based on model size and network fabric capabilities.

Advanced implementations address these challenges by employing adaptive strategies that overlap communication with computation and utilize tree-based drafting structures to increase the expected number of accepted tokens per iteration. Techniques such as dynamic batch sizing and confidence-aware early exiting further refine the trade-off between drafting length and verification success rates. By integrating these mechanisms, multi-device frameworks aim to saturate available hardware resources without compromising generation quality, offering a scalable solution for low-latency inference in distributed environments where single-device memory or compute limits are prohibitive [38].

4.4.2 Communication-Efficient Logit Transmission

In distributed large language model serving, the transmission of logits between devices often constitutes a critical latency bottleneck, particularly when model weights are sharded across multiple nodes. Traditional autoregressive decoding exacerbates this issue by requiring frequent synchronization after every single token generation, leading to underutilized bandwidth and high arithmetic intensity constraints. Speculative decoding mitigates this by trading compute for bandwidth, allowing the parallel generation of short token continuations where the forward pass latency for multiple tokens approximates that of a single token [39]. This approach significantly reduces the frequency of inter-device communication, effectively masking transmission overheads during the verification phase.

To further optimize communication efficiency, recent advancements employ adaptive mechanisms that dynamically adjust speculation lengths based on real-time traffic characteristics and model confidence. By analyzing request intervals and coefficient of variation, systems can select optimal draft sizes; for instance, sparse traffic benefits from longer speculation sequences to maximize parallelism, while intense traffic requires shorter sequences to minimize rollback costs. Ad-

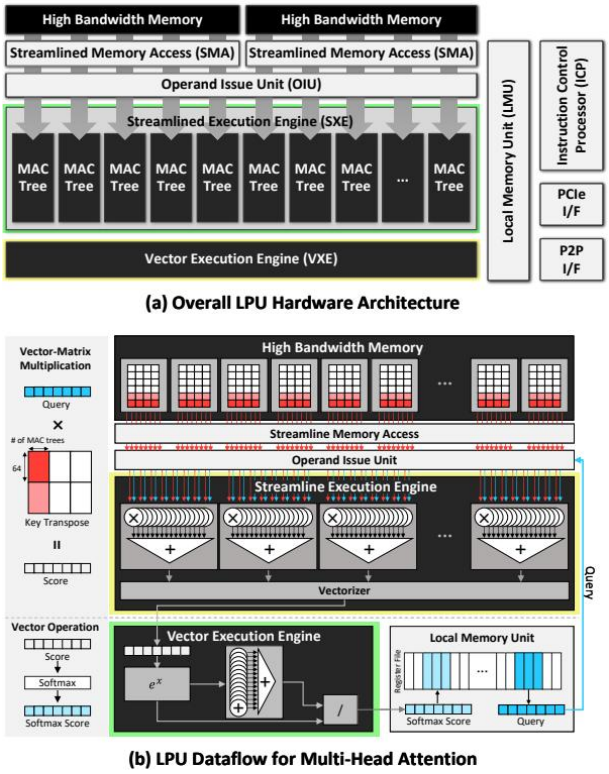

Figure 37: Chart from 'LPU: A Latency-Optimized and Highly Scalable Processor for Large Language Model Inference' (arXiv:2408.07326)

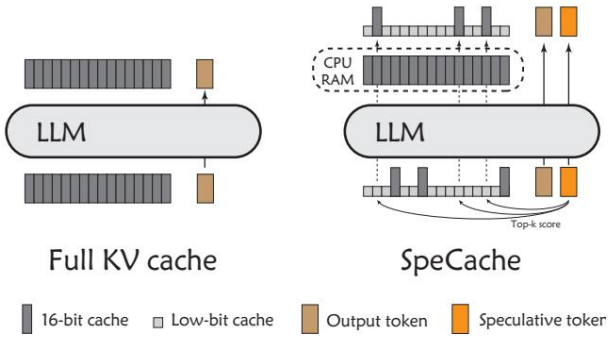

Figure 38: Chart from 'SpeCache: Speculative Key-Value Caching for Efficient Generation of LLMs' (arXiv:2503.16163)

ditionally, normalization techniques, such as scaling logit margins via sigmoid functions, enable precise adaptive control over drafting and verification. This adaptivity not only reduces wasted computation in high-uncertainty regions but also maximizes throughput in high-confidence scenarios, ensuring robust performance across diverse workloads without compromising output fidelity.

Beyond adaptive speculation, architectural innovations like prompt chunking and stochastic compression offer substantial reductions in communication overhead. Partitioning models into local and cloud components allows for the transmission of compressed latent representations rather than full floating-point embeddings, drastically lowering bit-rate requirements while maintaining privacy and utility. Furthermore, overlapping the transmission of chunked hidden states with in-cloud computation effectively hides communication delays, which otherwise scale linearly with prompt length. These strategies collectively transform the decoding process from a bandwidth-bound operation into a compute-efficient pipeline, achieving significant end-to-end speedups and enabling scalable deployment of massive models on heterogeneous hardware infrastructure [40].

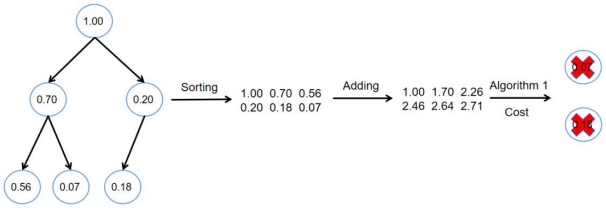

Figure 39: Chart from 'Inference-Cost-Aware Dynamic Tree Construction for Efficient Inference in Large Language Models' (arXiv:2510.26577)

4.4.3 Privacy-Preserving Split Inference

Privacy-preserving split inference addresses the critical tension between leveraging powerful cloud-based large language models and maintaining strict data confidentiality on user devices. This paradigm typically employs a U-shaped architecture where shallow input layers and deep output layers reside on the client, while computationally intensive middle layers are offloaded to the server. By keeping raw token embeddings and final generated text local, this approach prevents sensitive user data from traversing the network, thereby mitigating privacy leaks inherent in standard cloud inference. However, this partitioning introduces significant communication overhead and potential utility degradation due to the separation of model components.

To mitigate these challenges, recent frameworks integrate differential privacy mechanisms directly into the split inference pipeline. Specifically, user-side token embeddings are projected into low-dimensional latent spaces and subjected to differentially private stochastic quantization before transmission. This process ensures rigorous privacy guarantees by adding calibrated noise to the intermediate representations, effectively obfuscating individual data points while preserving the statistical properties necessary for accurate inference. Furthermore, server-side adaptations, such as soft prompt tuning, are employed to compensate for the utility loss caused by privacy-preserving perturbations and quantization errors, ensuring that the global model performance remains robust despite the distributed and noisy nature of the computation.

Beyond privacy mechanisms, optimizing the efficiency of split inference remains a primary focus to reduce latency and bandwidth consumption. Techniques such as adaptive speculation lengths and communication-efficient encoding schemes are utilized to minimize the number of interaction rounds between the client and server. By carefully balancing the computational load between local devices and remote servers, these methods aim to achieve near-real-time responsiveness without compromising the end-to-end privacy guarantees. Consequently, privacy-preserving split inference emerges as a viable solution for deploying personalized, secure, and efficient large language model services in resource-constrained and privacy-sensitive environments.

5 Future Directions

Despite significant advancements in KV cache optimization, current efficient serving techniques face persistent limitations regarding the trade-off between aggressive compression and generative fidelity. Existing mixed-precision and quantization strategies often incur substantial overhead from metadata management and irregular memory access patterns, particularly when handling statistical outliers that resist low-bit representation. Furthermore, dynamic token eviction policies, while effective for reducing memory footprints, risk prematurely discarding context essential for complex, multi-turn reasoning tasks, leading to coherence degradation in long-context scenarios. A critical gap remains in the holistic integration of these disparate methods; most systems optimize quantization, sparsity, and speculative decoding in isolation, failing to exploit the synergistic potential of co-designing algorithmic compression with hardware-specific constraints. Consequently, achieving stable, ultra-low-bit inference across diverse model architectures and varying context lengths without extensive per-model calibration remains an unresolved challenge.

Future research should prioritize the development of unified, adaptive frameworks that dynamically co-optimize quantization granularity, structural sparsity, and memory management policies in real time. Instead of static, offline calibration, next-generation systems must employ online sensitivity analysis to adjust bit-widths and eviction thresholds based on the immediate semantic complexity of the input stream. This includes investigating learnable compression mechanisms where the model itself predicts optimal cache states, effectively merging the compression policy with the generative process. Additionally, there is a pressing need for hardware-aware algorithmic co-design that moves beyond generic GPU optimizations to target emerging accelerator architectures, such as near-memory computing units and specialized integer-only tensor cores. Exploring hybrid numerical formats that seamlessly switch between integer and low-precision floating-point representations within a single inference pass could further mitigate outlier impacts while maximizing throughput. Finally, rigorous theoretical bounds on information loss during aggressive cache pruning must be established to guarantee reliability in safety-critical applications.

The realization of these future directions promises to fundamentally transform the scalability and accessibility of large language models. By overcoming current memory bandwidth bottlenecks, advanced serving techniques will enable the deployment of massive models on resource-constrained edge devices, democratizing access to high-level AI capabilities without reliance on cloud infrastructure. Moreover, efficient cache management will drastically reduce the energy consumption and latency of production systems, making real-time, long-context interaction economically viable for widespread commercial adoption. Ultimately, bridging the gap between theoretical compression limits and practical system efficiency will unlock new application domains requiring extensive context windows and continuous learning, driving the next wave of innovation in autonomous agents and complex reasoning systems.

6 Conclusion

This survey has provided a comprehensive synthesis of efficient serving techniques for Large Language Models, with a particular emphasis on optimizing the Key-Value cache to alleviate memory bandwidth bottlenecks. We systematically examined three pivotal paradigms: quantization and compression, structural sparsity exploitation, and speculative decoding integration. Our analysis revealed that mixed-precision allocation and outlier mitigation via smoothing techniques effectively balance memory efficiency with generative fidelity by addressing the asymmetric distribution of activation values. Furthermore, we demonstrated how integer-only arithmetic frameworks enable deployment on resource-constrained hardware by eliminating floating-point operations without significant accuracy degradation. The review also highlighted the critical role of attention support awareness and dynamic token eviction policies, which leverage intrinsic sparsity to selectively retain salient information while discarding redundant entries. Finally, we explored advanced methodologies rooted in low-rank decomposition and codebook learning, illustrating how these approaches achieve ultra-low bit-width representations while preserving long-range dependencies essential for coherent generation.

The significance of this work lies in its unified taxonomy that bridges the gap between theoretical compression algorithms and practical system-level

implementations. Unlike previous reviews that often treat model pruning, distillation, and quantization in isolation, this survey elucidates the intricate interactions between KV cache optimization strategies and speculative decoding mechanisms. By critically evaluating these techniques across diverse hardware architectures and context lengths, we have identified key trends that define the current state of efficient inference. This holistic perspective offers researchers and practitioners a robust foundation for understanding the inherent trade-offs between latency, throughput, and model accuracy. Consequently, this review serves as an essential reference for designing next-generation serving systems capable of scaling to the demands of complex, multi-turn conversations and extensive document processing without prohibitive memory costs.

Looking forward, the field must address several unresolved challenges to fully realize the potential of efficient LLM serving. Future research should focus on developing adaptive, hardware-aware frameworks that dynamically adjust compression strategies in real-time based on workload characteristics and available resources. There is a pressing need for standardized benchmarks that evaluate not only accuracy but also energy efficiency and end-to-end latency across heterogeneous computing environments. Moreover, the integration of emerging memory technologies with algorithmic innovations promises to further reduce the footprint of massive models. We call upon the community to foster deeper collaboration between algorithm designers and system architects to co-optimize software and hardware layers. By advancing these frontiers, we can ensure that large-scale language models remain accessible, scalable, and sustainable for a broad range of applications, ultimately democratizing access to advanced artificial intelligence capabilities.

References

[1] HADES Hardware Accelerated Decoding for Efficient Speculation in Large Language Models

[2] SAW-INT4 System-AWare 4-Bit KV-Cache Quantization for Real-World LLM Serving

[3] FastCache Optimizing Multimodal LLM Serving through Lightweight KV-Cache Compression Framework

[4] EVICPRESS Joint KV-Cache Compression and Eviction for Efficient LLM Serving

[5] Cocktail Chunk-Adaptive Mixed-Precision Quantization for Long-Context LLM Inference

[6] Designing Efficient LLM Accelerators for Edge Devices

[7] Efficient LLM Inference on CPUs

[8] SpecEE Accelerating Large Language Model Inference with Speculative Early Exiting

[9] Progressive Sparse Attention Algorithm and System Co-design for Efficient Attention in LLM Serving

[10] Lethe Layer- and Time-Adaptive KV Cache Pruning for Reasoning-Intensive LLM Serving

[11] KEYFORMER KV CACHE REDUCTION THROUGH KEY TOKENS SELECTION-FOR EFFICIENT GENERATIVE INFERENCE

[12] FLEXICACHE LEVERAGING TEMPORAL STABILITY OF ATTENTION HEADS FOR EFFICIENT KV CACHE MANAGEMENT

[13] ReLU Strikes Back Exploiting Activation Sparsity in Large Language Models

[14] LUT-LLM Efficient Large Language Model Inference with Memory-based Computations on FPGAs

[15] Inference Optimization of Foundation Models on AI Accelerators

[16] PHOTON Hierarchical Autoregressive Modeling for Lightspeed and Memory-Efficient Language Generation

[17] AdaptCache KV Cache Native Storage Hierarchy for Low-Delay and High-Quality Language Model Serving

[18] Efficient Memory Management for Large Language Model Serving with PagedAttention

[19] Strata Hierarchical Context Caching for Long Context Language Model Serving

[20] MoQAE Mixed-Precision Quantization for Long-Context LLM Inference via Mixture of Quantization-Aware Experts

[21] eLLM Elastic Memory Management Framework for Efficient LLM Serving

[22] KVShare An LLM Service System with Efficient and Effective Multi-Tenant KV Cache Reuse

[23] OrbitFlow SLO-Aware Long-Context LLM Serving with Fine-Grained KV Cache Reconfiguration

[24] TraCT Disaggregated LLM Serving with CXL Shared Memory KV Cache at Rack-Scale
[25] ObjectCache Layerwise Object-Storage Retrieval for KV Cache Reuse
[26] Serve Programs, Not Prompts
[27] Efficient and Workload-Aware LLM Serving via Runtime Layer Swapping and KV Cache Resizing
[28] A Comprehensive Survey of Accelerated Generation Techniques in Large Language Models
[29] CLLMs Consistency Large Language Models
[30] Abstract
[31] A Survey on Efficient Inference for Large Language Models
[32] FASTDECODE High-Throughput GPU-Efficient LLM Serving using Heterogeneous Pipelines
[33] Quasar Quantized Self-Speculative Acceleration for Rapid Inference via Memory-Efficient Verification
[34] SpecMemo Speculative Decoding is in Your Pocket
[35] CARD Cache-Assisted Parallel Speculative Decoding for Efficient Large Language Model Inference
[36] Confidence-Modulated Speculative Decoding for Large Language Models
[37] Nightjar Dynamic Adaptive Speculative Decoding for Large Language Models Serving
[38] Contemporary Industry Products
[39] SpeCache Speculative Key-Value Caching for Efficient Generation of LLMs
[40] INFERENCE-COST-AWARE DYNAMIC TREE CONSTRUC-TION FOR EFFICIENT INFERENCE IN LARGE LANGUAGE-MODELS